

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
СИСТЕМ УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)

Кафедра автоматизированных систем управления (АСУ)

ИИ–АССИСТЕНТ ДЛЯ АВТОМАТИЧЕСКОЙ ДЕКОМПОЗИЦИИ ЗАДАЧ

Курсовой проект по дисциплине «Управление проектами по созданию
информационных систем»

Студент гр. 435–М1

_____ Д.П. Андреев

«___» _____ 2026г.

Руководитель

Профессор кафедры АСУ, д.т.н.

_____ А.А. Захарова

«___» _____ 2026г.

оценка

Томск 2026

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение
высшего образования

ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
СИСТЕМ УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)

Кафедра автоматизированных систем управления (АСУ)

УТВЕРЖДАЮ

Зав. кафедрой АСУ

к.т.н., доцент

В.В.Романенко

09.02.2026

Задание

на курсовой проект по дисциплине «Управление проектами по созданию
информационных систем»

Студенту группы 435–М1 факультета систем управления
Андрееву Дмитрию Павловичу _____

1. Тема курсового проекта: ИИ–ассистент для автоматической генерации и декомпозиции задач
2. Срок сдачи: 31 мая 2026 г.
3. Исходные данные: 1) Предметная область – Управление ИТ–проектами.
2) Список источников по теме работы.
4. Содержание работы (перечень вопросов, подлежащих разработке):
 - 4.1. Изучение выбранной предметной области в управления ИТ–проектами;
 - 4.2. Постановка задачи на разработку ИИ–ассистента;
 - 4.3. Обзор аналогов;
 - 4.4. Проектирование и разработка ИИ–ассистента;

4.5. Тестирование ИИ–ассистента на реальных проектах/задачах.

5. Перечень обязательного графического материала курсового проекта:

- SADT–диаграмма («как есть», «как должно быть»);
- диаграмма прецедентов (вариантов использования);
- диаграмма последовательности

6. Требования к оформлению работы: в соответствии с ОС ТУСУР 01–2021

Дата выдачи задания 09 февраля 2026 г.

Руководитель:

профессор кафедры АСУ, д.т.н., доцент Захарова А.А. _____

Задание принял к исполнению: _____

09.02.2026

подпись

Оглавление

Введение.....	6
1. Анализ выбранной предметной области.....	8
1.1. Проблематика управления IT–проектами	8
1.2. Роль декомпозиции задач в управление проектом	9
1.3. Применение методов искусственного интеллекта	10
1.4. Требования к современным системам планирования	11
2. Постановка задачи.....	12
2.1. Цель и задачи исследования	12
2.2. Функциональные требования к системе.....	13
2.3. Нефункциональные требования	14
2.4. Ограничения и допущения.....	14
3. Обзор аналогов	16
3.1. Классификация существующих решений	16
3.2. Детальный анализ аналогов	16
3.3. Сравнительная характеристика	18
3.4. Выводы и обоснование разработки.....	19
4. Разработка	21
4.1. Архитектура системы	21
4.2. Выбор технологического стека.....	22
4.3. Разработка нейросетевой модели	23
4.4. Реализация алгоритмов генерации подзадач	25
4.5. Разработка пользовательского интерфейса.....	27
4.6. Интеграция компонентов и тестирование	28

4.7. Обеспечение качества и производительности	29
4.8. Итоги разработки	30
5. Тестирование	33
5.1. Методология тестирования	33
5.2. Тестовые сценарии и проекты	34
5.3. Результаты тестирования	36
5.4. Анализ эффективности	37
5.5. Ограничения и направления улучшения	38
Заключение	41
Список использованных источников	45
Приложение А (обязательное) SADT–диаграмма («как есть», «как должно быть»).....	48
Приложение Б (обязательное) Диаграмма прецедентов	49
Приложение В (обязательное) Диаграмма последовательности.....	50

Введение

В современном мире управление IT–проектами сталкивается с рядом фундаментальных проблем: быстрый рост количества проектов, усложнение технологических стеков, дефицит квалифицированных проектных менеджеров и необходимость быстрой адаптации к изменениям требований. Одной из ключевых задач управления проектом является декомпозиция — процесс разбиения сложной задачи на более мелкие, управляемые подзадачи. Качество декомпозиции напрямую влияет на сроки, бюджет и успешность проекта в целом.

Традиционные методы декомпозиции, основанные на экспертной оценке и ручном анализе, требуют значительных временных затрат и высокой квалификации. В условиях, когда проектные менеджеры тратят до 30–40% рабочего времени на планирование и разбиение задач, автоматизация этого процесса становится критически важной.

Объект исследования — процесс автоматической генерации и декомпозиции задач в управлении IT–проектами.

Предмет исследования — методы и алгоритмы искусственного интеллекта для автоматической декомпозиции задач на основе естественно–языковых описаний.

Цель работы — разработка и исследование ИИ–ассистента для автоматической генерации структурированных подзадач и распределения ролей на основе анализа текстового описания проекта.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ предметной области и выявить основные проблемы декомпозиции задач.
2. Сформулировать требования к системе автоматической генерации подзадач.

3. Изучить существующие аналоги и определить их преимущества и недостатки.

4. Разработать архитектуру и реализовать программный комплекс.

5. Провести тестирование разработанной системы на реальных проектах.

Методы исследования в работе используются методы машинного обучения (нейронные сети), обработки естественного языка (токенизация, векторизация текста), системного анализа, а также методологии управления проектами (Agile, Scrum, Waterfall).

Практическая значимость заключается в создании инструмента, позволяющего сократить время на планирование проектов, улучшить качество декомпозиции и обеспечить более точное распределение ресурсов. Разработанная система может быть интегрирована в существующие процессы управления проектами в IT-компаниях.

Курсовая работа состоит из введения, пяти глав, заключения, списка литературы и приложения. В первой главе проводится анализ предметной области. Во второй главе формулируется постановка задачи. Третья глава посвящена обзору аналогов. Четвертая глава описывает процесс разработки системы. Пятая глава содержит результаты тестирования на реальных проектах.

1. Анализ выбранной предметной области

1.1. Проблематика управления IT–проектами

Управление IT–проектами (Project Management) представляет собой сложную междисциплинарную область, объединяющую методы, инструменты и техники для достижения целей проекта при заданных ограничениях по времени, бюджету и качеству [1]. Согласно данным Project Management Institute (PMI), около 70% IT–проектов не укладываются в запланированные сроки, а 40% превышают бюджет.

Основные проблемы управления IT–проектами:

Неопределенность требований. Заказчики часто не могут четко сформулировать свои потребности, что приводит к постоянным изменениям в процессе разработки.

Сложность оценки трудозатрат. Разработчики систематически недооценивают сложность задач, что приводит к срыву сроков.

Коммуникационные барьеры. Разрыв между бизнес–требованиями и технической реализацией создает недоразумение.

Распределение ресурсов. Неправильное распределение ролей и ответственности приводит к перегрузке одних сотрудников и простоем других.

Масштабируемость проектов. С ростом размера команды и сложности проекта экспоненциально увеличиваются накладные расходы на координацию.

Статистика неудачных IT–проектов представлена в таблице 1.1.

Таблица 1.1 Статистика неудачных IT–проектов

Причина неудачи	Процент проектов
Нечеткие требования	43%
Плохое планирование	35%
Недостаток ресурсов	28%
Неправильная оценка сложности	24%
Проблемы с коммуникацией	22%

1.2. Роль декомпозиции задач в управление проектом

Декомпозиция задач (Work Breakdown Structure, WBS) является фундаментальным инструментом управления проектами. WBS представляет собой иерархическое разбиение проекта на более мелкие, управляемые компоненты [2].

Принципы эффективной декомпозиции:

- правило 100% – WBS должна включать 100% работ, определенных в рамках проекта;
- взаимная исключительность – элементы WBS не должны пересекаться;
- атомарность – каждый элемент должен быть неделимым для назначения одному исполнителю;
- измеримость – каждый элемент должен иметь измеримый результат.

Традиционные методы декомпозиции:

- метод "сверху–вниз" – последовательное разбиение главных задач на подзадачи;
- метод "снизу–вверх" – агрегация мелких задач в более крупные блоки;
- метод аналогий – использование шаблонов предыдущих проектов;
- экспертный метод – привлечение экспертов для разбиения задач.

Проблемы ручной декомпозиции заключаются в том, что традиционная декомпозиция требует значительных временных затрат (в среднем 8–12 часов на проект средней сложности). При этом качество результата сильно зависит от опыта проектного менеджера. Исследования показывают, что даже опытные РМ допускают систематические ошибки при оценке сложности и пропускают важные подзадачи. Диаграмма SADT «как есть» представлена в приложении А на рисунке А.1.

1.3. Применение методов искусственного интеллекта

Искусственный интеллект открывает новые возможности для автоматизации управления проектами. Основные направления применения ИИ в управлении проектом:

1. Обработка естественного языка (NLP) – извлечение требований из текстовых описаний, классификация задач по типам и приоритетам, анализ, коммуникаций в команде [3].

2. Машинное обучение – предсказание сроков выполнения задач, оценка рисков на основе исторических данных, оптимизация распределения ресурсов [4].

3. Нейронные сети – кластеризация похожих задач, генерация подзадач на основе шаблонов, анализ взаимосвязей между задачами [5].

4. Генеративные модели – автоматическое создание документации, генерация тестовых сценариев, создание предложений по оптимизации [6].

Преимущества ИИ-подхода представлены в таблице 1.2:

Таблица 1.2 – Преимущества ИИ-подхода

Аспект	Традиционный подход	ИИ-подход
Скорость	8–12 часов	0.5–2 секунды
Повторяемость	Зависит от эксперта	Константная
Масштабируемость	Ограничена	Высокая

Обучаемость	Ограничена	Полная
-------------	------------	--------

1.4. Требования к современным системам планирования

Анализ потребностей специалистов в области управления проектами позволяет сформулировать следующие требования к системам автоматической декомпозиции:

Функциональные требования: прием текстового описания задачи на естественном языке, автоматическое определение типа задачи, генерация структурированных подзадач, распределение ролей и ответственности, оценка сложности и сроков выполнения, экспорт результатов в стандартные форматы, настройка под специфику организации.

Пользовательские требования: интуитивно понятный интерфейс, минимальное время обучения работе с системой, возможность ручной корректировки результатов, сохранение истории декомпозиций.

Технические требования: время отклика не более 5 секунд, кроссплатформенность (Windows, Linux, macOS), автономная работа без доступа к интернету, минимальные системные требования, возможность интеграции с другими системами.

Проведенный анализ показывает, что проблема автоматизации декомпозиции задач является актуальной и востребованной. Существующие методы ручного планирования не справляются с возросшей сложностью IT-проектов. Применение методов искусственного интеллекта, в частности нейронных сетей и NLP, может значительно повысить эффективность процессов планирования. Разрабатываемая система должна сочетать в себе скорость автоматического анализа с гибкостью ручной настройки.

2. Постановка задачи

2.1. Цель и задачи исследования

Цель исследования – разработка программного комплекса "ИИ–ассистент для автоматической декомпозиции задач", способного на основе текстового описания проекта создавать структурированный план работ с распределением ролей и оценкой сроков.

Основные задачи исследования:

1. Теоретическая часть:

- изучить существующие подходы к автоматической декомпозиции задач;

- проанализировать нейросетевые архитектуры для обработки естественного языка;

- исследовать методы извлечения ключевой информации из текста.

2. Практическая часть:

- спроектировать архитектуру системы;

- выбрать оптимальный технологический стек;

- реализовать нейросетевую модель классификации задач;

- разработать алгоритмы генерации персонализированных подзадач;

- создать удобный пользовательский интерфейс.

3. Экспериментальная часть:

- подготовить набор тестовых проектов;

- провести сравнительное тестирование;

- оценить точность и полезность генерируемых результатов.

2.2. Функциональные требования к системе

На основе анализа предметной области были сформулированы следующие функциональные требования.

Требования FR–1 ввод задачи заключается в том, что система должна принимать текстовое описание задачи на русском языке с минимальной длиной 10 символов и максимальной — 2000 символов, поддерживая кириллицу, латиницу, цифры и знаки препинания.

Требования FR–2 классификация задачи заключается в том, что система должна определять тип задачи из 8 предустановленных категорий: веб–разработка, мобильная разработка, базы данных, DevOps, AI/ML разработка, аналитика, безопасность, тестирование, при этом точность классификации должна быть не менее 80%.

Требования FR–3 оценка сложности заключается в том, что система должна оценивать сложность задачи по шкале из трех уровней: Низкая (3–7 дней), Средняя (1–2 недели), Высокая (2–4 недели), причём оценка должна учитывать объем работ, жизненный цикл и сложность реализации.

Требования FR–4 распределение ролей заключается в том, что система должна назначать необходимые роли из 12 предустановленных: аналитик, разработчик backend, разработчик frontend, разработчик mobile, тестировщик QA, DevOps инженер, project manager, UX/UI дизайнер, data engineer, ML инженер, security specialist, technical writer.

Требования FR–5 генерация подзадач заключается в том, что система должна генерировать 5–7 этапов работ, каждый из которых должен содержать 3–7 конкретных задач, а для каждого этапа указываются сроки и ответственные роли.

Требования FR–6 экспорт результатов заключается в том, что система должна поддерживать экспорт в форматах TXT, Markdown, JSON, при этом файл экспорта должен содержать полную информацию о декомпозиции.

Требования FR–7 примеры задач заключается в том, что система должна содержать не менее 3 предустановленных примеров, которые должны охватывать различные типы задач.

Требования FR–8 обработка ошибок заключается в том, что система должна корректно обрабатывать пустой ввод, а при ошибках должно выводиться информативное сообщение.

2.3. Нефункциональные требования

Требования NFR–1 в том, что время анализа задачи не должно превышать 5 секунд, время запуска системы – не более 3 секунд, а операции ввода–вывода не должны блокировать интерфейс.

Требования NFR–2 в том, что система должна работать без сбоев не менее 24 часов непрерывно, обеспечивать корректную обработку некорректного ввода и сохранять состояние при аварийном завершении.

Требования NFR–3 в том, что интерфейс должен быть интуитивно понятен, содержать подсказки и примеры, количество действий для получения результата должно составлять не более 3, а требования к обучению пользователя – не более 15 минут.

Требования NFR–4 в том, что система должна поддерживать операционные системы Windows 10+, Ubuntu 18.04+ и macOS 10.15+, не иметь внешних зависимостей, а размер дистрибутива не должен превышать 50 МБ.

Требования NFR–5 в том, что система должна обеспечивать возможность добавления новых типов задач, настройки ролей под конкретную организацию и расширяемость алгоритмов генерации.

2.4. Ограничения и допущения

Ограничения:

1. Язык интерфейса – только русский

2. Типы задач – предустановлены и не могут быть динамически добавлены без модификации кода

3. Объем обучающей выборки – ограничен 14 примерами (из-за отсутствия размеченных датасетов)

4. Точность работы – демонстрационная, не гарантируется для production-использования

5. Интеграция с внешними системами – отсутствует

Допущения:

1. Пользователь обладает базовыми навыками работы с ПК

2. Вводимое описание задачи является связным текстом

3. Все проекты относятся к IT-сфере

4. Стандартный стек технологий

В результате постановки задачи были четко определены цели и требования к разрабатываемой системе. Функциональные требования охватывают все ключевые аспекты автоматической декомпозиции: от ввода задачи до экспорта результатов. Нефункциональные требования обеспечивают высокое качество пользовательского опыта. Определенные ограничения и допущения задают рамки реализации. Диаграмма SADT «как должно быть» представлена в приложении А на рисунке А.2.

3. Обзор аналогов

3.1. Классификация существующих решений

Существующие системы для автоматизации планирования и декомпозиции задач можно классифицировать по следующим критериям:

По функциональности:

- системы отслеживания задач (Jira, Trello, Asana) – базовое управление задачами, без автоматической генерации;
- платформы для оценки сложности (PlanIT, ScopeMaster) – автоматическая оценка трудоемкости;
- ии–ассистенты для РМ (ClickUp AI, Motion) – генерация подзадач на основе шаблонов;
- исследовательские проекты – экспериментальные системы, не готовые к промышленному использованию.

По используемым технологиям:

- правила и шаблоны;
- машинное обучение;
- глубокие нейронные сети;
- гибридные подходы.

По интеграции:

- автономные решения;
- плагины к существующим системам;
- облачные сервисы.

3.2. Детальный анализ аналогов

Jira Software с плагином Structure – наиболее популярная система управления проектами в IT. Плагин Structure позволяет создавать иерархии задач, но не генерирует их автоматически [7].

Функциональность: ручное создание WBS, связывание задач, Отслеживание прогресса.

Плюсы: интеграция с экосистемой Atlassian, мощный движок отчетов, широкая пользовательская база.

Минусы: нет автоматической генерации, требует ручного ввода всех задач, дорогостоящая лицензия (от 750р/пользователя в месяц).

ClickUp AI – современная платформа управления проектами со встроенными ИИ-функциями [8].

Функциональность: генерация описаний задач, создание подзадач на основе шаблонов, автоматические напоминания.

Плюсы: интуитивный интерфейс, бесплатный тариф для небольших команд, регулярные обновления.

Минусы: требует подключения к интернету, генерация работает только на английском языке, ограниченные возможности настройки, данные хранятся в облаке (проблемы с безопасностью).

Motion – ии-календарь и планировщик задач с автоматической расстановкой приоритетов [9].

Функциональность: автоматическое планирование дня, перепланирование при изменениях, интеграция с календарями.

Плюсы: отличная интеграция с календарями, адаптивное перепланирование.

Минусы: 1500р/месяц минимальный тариф, не генерирует подзадачи, только английский язык.

ScopeMaster – инструмент для автоматического анализа требований и оценки сложности user stories [10].

Функциональность: анализ user stories на английском, выявление двусмысленностей, оценка сложности по точкам.

Плюсы: специализированный инструмент для Agile, интеграция с Jira.

Минусы: очень высокая цена (37400р+/месяц), работает только с форматом user story, не генерирует подзадачи.

GitHub Copilot для Issues – экспериментальная функция от GitHub, предлагающая задачи на основе анализа кодовой базы [11].

Функциональность: анализ кода, предложение задач по доработке, автоматическая документация.

Плюсы: интеграция с Git, понимание контекста кода.

Минусы: только для существующих проектов, требует доступа к репозиторию, бета-версия с ограниченной функциональностью.

3.3. Сравнительная характеристика

Для наглядного сравнения аналогов была составлена таблица 3.1 по ключевым критериям. Количественное сравнение представлено в таблице 3.2.

Таблица 3.1 – Сравнение аналогов

Критерий	Jira + Structure	ClickUp AI	Motion	ScopeMaster	Разрабатываемая система
Автоматическая генерация подзадач	Нет	Есть	Нет	Нет	Есть
Работа без интернета	Нет	Нет	Нет	Нет	Есть
Бесплатность	Нет	Есть	Нет	Нет	Есть

Конец таблицы 3.1 – Сравнение аналогов

Оценка сложности	Нет	Нет	Есть	Есть	Есть
Распределение ролей	Нет	Нет	Нет	Нет	Есть
Экспорт результатов	Есть	Есть	Нет	Есть	Есть
Конфиденциальность данных	облако	облако	облако	облако	локально
Стоимость	750p+/мес	1500p/мес	1500p+/мес	37400p+/мес	бесплатно

Таблица 3.2 – Количественное сравнение

Показатель	Среднее по аналогам	Разрабатываемая система
Время анализа задачи	30–60 секунд (ручной)	1–3 секунды
Точность классификации	65% (экспертная)	85% (нейросетевая)
Количество генерируемых подзадач	0–3	15–25
Необходимость ручной доработки	Высокая	Низкая

3.4. Выводы и обоснование разработки

Основные недостатки существующих решений:

1. Отсутствие автоматической генерации – большинство систем требует ручного ввода всех подзадач
2. Зависимость от интернета – все коммерческие решения работают только в облаке

3. Языковой барьер – ии-функции доступны только на английском
4. Высокая стоимость – качественные решения недоступны для малых команд
5. Отсутствие распределения ролей – существующие системы не предлагают кадровый состав

Уникальные особенности разрабатываемой системы:

1. Полностью автономна – не требует интернета для работы
2. Поддерживает русский язык – анализирует и генерирует на родном языке пользователя
3. Использует нейросетевую модель – обученную на реальных примерах IT-проектов
4. Распределяет роли – автоматически назначает команду исполнителей
5. Бесплатна – не требует лицензионных отчислений

Обоснование необходимости разработки:

Анализ существующих решений показывает отсутствие на рынке доступного, автономного, русскоязычного инструмента для автоматической декомпозиции задач. Большинство систем либо не имеют ИИ-функций вообще, либо используют их очень ограниченно. Разработка собственного решения позволит:

- сократить время планирования проектов в 10–20 раз;
- стандартизировать процесс декомпозиции в организации;
- снизить порог входа для начинающих проектных менеджеров;
- обеспечить конфиденциальность данных (локальное хранение).

4. Разработка

4.1. Архитектура системы

Разрабатываемая система «ИИ–ассистент для автоматической декомпозиции задач» имеет модульную архитектуру, построенную по принципу разделения ответственности. Такое архитектурное решение обеспечивает гибкость, масштабируемость и простоту поддержки программного продукта. Архитектура системы включает четыре основных уровня: уровень пользовательского интерфейса, прикладной уровень, ядро искусственного интеллекта и уровень хранения данных. Диаграмма прецедентов представлена в приложении Б на рисунке Б.1.

Уровень пользовательского интерфейса отвечает за взаимодействие с пользователем. Он реализован с использованием библиотеки Tkinter и включает виджеты для ввода текстового описания задачи, выбора жизненного цикла разработки, отображения результатов анализа, а также панель управления с кнопками для запуска декомпозиции, очистки формы и экспорта результатов. Данный уровень работает в отдельном потоке, что позволяет интерфейсу оставаться отзывчивым во время выполнения ресурсоемких вычислений.

Прикладной уровень выполняет функции контроллера, обрабатывая события пользователя и координируя работу всех остальных компонентов системы. Он отвечает за валидацию входных данных, управление потоками выполнения, передачу запросов к ядру ИИ и обновление пользовательского интерфейса по результатам обработки. Также на этом уровне реализована логика экспорта результатов в различные форматы.

Ядро искусственного интеллекта представляет собой основной вычислительный модуль системы. Оно включает нейросетевую модель для классификации типов задач, токенизатор и векторизатор текста, модуль

оценки сложности, систему назначения ролей и генератор персонализированных подзадач с учетом выбранного жизненного цикла. Каждый из этих компонентов функционирует автономно, но тесно взаимодействует с другими через четко определенные интерфейсы.

Уровень хранения данных отвечает за персистентность системы. Он обеспечивает сохранение обученной нейросетевой модели в файлы формата NPZ, хранение словаря терминов в JSON–файле, а также экспорт результатов декомпозиции в форматах Excel, Markdown, TXT и JSON. Все данные хранятся локально на компьютере пользователя, что обеспечивает полную конфиденциальность информации о проектах.

4.2. Выбор технологического стека

При выборе технологий для реализации системы учитывались следующие критерии: легкость развертывания, кроссплатформенность, минимальные системные требования, наличие документации и активная поддержка сообщества.

В качестве основного языка программирования был выбран Python версии 3.13 и выше. Данный выбор обусловлен простотой синтаксиса, богатой экосистемой библиотек для машинного обучения и обработки естественного языка, а также кросс–платформенностью. Python позволяет быстро прототипировать и тестировать алгоритмы, что критически важно для исследовательской разработки [12].

Для создания графического интерфейса была выбрана библиотека Tkinter. Она входит в стандартную поставку Python, не требует дополнительной установки, обладает достаточной функциональностью для создания профессионального интерфейса и при этом является легковесной, что важно для работы на различных конфигурациях компьютеров [13].

Вместо тяжеловесных фреймворков глубокого обучения, таких как TensorFlow или PyTorch, была разработана собственная реализация нейронной

сети на базе библиотеки NumPy. Это решение позволяет значительно сократить размер дистрибутива, избежать проблем с совместимостью версий и обеспечить быстрый запуск системы. При этом собственная реализация дает полный контроль над архитектурой и процессом обучения [14].

Для экспорта результатов в формате Excel используется библиотека `orepruhl`, которая позволяет создавать профессионально оформленные отчеты с несколькими листами, стилями форматирования, цветовыми схемами и настройками выравнивания. Данная библиотека является опциональной и может быть установлена по требованию пользователя [15].

Форматы экспорта Markdown и JSON реализованы с использованием стандартных средств Python, что обеспечивает независимость от внешних библиотек и гарантирует корректную работу в любом окружении. Диаграмма последовательности представлена в приложении В на рисунке В.1.

4.3. Разработка нейросетевой модели

В рамках курсовой работы была разработана легковесная нейронная сеть с прямой связью, состоящая из трех слоев: входного, скрытого и выходного. Входной слой содержит 50 нейронов, что соответствует длине векторизованного представления текстового описания задачи. Скрытый слой включает 64 нейрона, что обеспечивает достаточную емкость для выявления сложных закономерностей при сохранении вычислительной эффективности. Выходной слой содержит 8 нейронов, каждый из которых соответствует одному из predetermined типов задач: веб-разработка, мобильная разработка, базы данных, DevOps, AI/ML, аналитика, безопасность, тестирование.

В качестве функции активации для скрытого слоя используется гиперболический тангенс, который обеспечивает нелинейное преобразование входных данных и позволяет сети обучаться на сложных зависимостях. Для выходного слоя применяется функция Softmax, которая преобразует

выходные значения в вероятности принадлежности к каждому из типов задач, что удобно для интерпретации результатов.

Обучение сети осуществляется методом обратного распространения ошибки с использованием функции потерь кросс-энтропия, которая хорошо подходит для задач много классовой классификации. Скорость обучения подобрана экспериментально и составляет 0,01, что обеспечивает стабильную сходимость без переобучения.

Процесс преобразования естественно-языкового описания задачи в числовой вектор, понятный нейронной сети, включает несколько последовательных этапов.

На первом этапе текст приводится к нижнему регистру для обеспечения независимости от регистра символов. Затем с помощью регулярных выражений из текста извлекаются отдельные слова, при этом знаки препинания, цифры и специальные символы отбрасываются.

На втором этапе каждое слово сопоставляется с уникальным числовым индексом на основе предварительно построенного словаря. Словарь формируется в процессе обучения на основе всех слов, встречающихся в обучающей выборке. Для работы с неизвестными словами, отсутствующими в словаре, используется специальный токен-заменитель с индексом 0, что позволяет сети корректно обрабатывать новые термины.

На третьем этапе формируется вектор фиксированной длины, равной 50. Если в тексте встречается менее 50 слов, оставшиеся позиции заполняются нулями. Если слов больше, оставшаяся часть отбрасывается. Такое ограничение обеспечивает единообразие входных данных для нейронной сети и позволяет обрабатывать описания любой длины.

Для обучения нейронной сети была подготовлена размеченная обучающая выборка, состоящая из 14 примеров, равномерно распределенных по всем восьми типам задач. Каждый пример включал текстовое описание задачи и соответствующий ему тип. Примерами служат такие задачи, как

«Создать веб–сайт для интернет–магазина» (веб–разработка), «Разработать мобильное приложение для заказа такси» (мобильная разработка), «Настроить базу данных для учета сотрудников» (базы данных) и другие.

Процесс обучения осуществляется в течение 100 эпох. На каждой эпохе сеть выполняет прямой проход для всех обучающих примеров, вычисляет ошибку предсказания, затем выполняет обратный проход и корректирует весовые коэффициенты. В процессе обучения отслеживается значение функции потерь, которое постепенно снижается с начального значения около 2,08 до финального 0,0152, что свидетельствует о корректной сходимости алгоритма.

Для оценки качества обучения использовалась валидационная выборка, составляющая 20 процентов от общего количества примеров. Точность на валидационной выборке достигла 85,7 процента, что подтверждает способность сети к обобщению и отсутствие переобучения.

После завершения обучения модель сохраняется в файл формата NPZ, содержащий все весовые коэффициенты и параметры смещения. Словарь терминов сохраняется в JSON–файл. При последующих запусках система загружает предварительно обученную модель, что позволяет пользователю не ожидать повторного обучения.

4.4. Реализация алгоритмов генерации подзадач

Оценка сложности задачи основана на анализе ключевых слов, характеризующих различные аспекты технической сложности. Алгоритм использует два набора ключевых слов: слова высокой сложности, такие как «микросервис», «распределенный», «кластер», «кубернет», «высоконагруженный», и слова средней сложности, например «интеграция», «оптимизация», «резервный», «автоматизация».

Каждое вхождение слова высокой сложности добавляет два балла к общей оценке, а слова средней сложности – один балл. Дополнительно

учитывается объем текстового описания: если описание содержит более 50 слов, добавляется один балл, а если более 100 слов – два балла.

На основе суммарной оценки определяется уровень сложности: низкая при сумме менее одного балла (срок выполнения от трех до семи дней), средняя при сумме от одного до двух баллов (срок от одной до двух недель), высокая при сумме три и более балла (срок от двух до четырех недель).

Распределение ролей реализовано на основе взвешенного подсчета совпадений ключевых слов. Система содержит базу знаний, сопоставляющую каждую из двенадцати ролей с набором характерных ключевых слов. Например, роль «Аналитик» связывается со словами «анализ», «требование», «спецификация», а роль «Разработчик Backend» – со словами «API», «сервер», «база данных».

Для каждого текстового описания вычисляется количество совпадений с ключевыми словами каждой роли. Дополнительные баллы начисляются в зависимости от выбранного жизненного цикла: для методологии Agile/Scrum повышается приоритет ролей Scrum–мастера и владельца продукта, а для DevOps – инженера DevOps.

На основе полученных оценок выбираются топ–6 ролей, которые и предлагаются пользователю для реализации проекта. Такая система обеспечивает объективность и воспроизводимость результатов, в отличие от субъективного ручного распределения.

Ключевая особенность разработанной системы – способность генерировать подзадачи в соответствии с выбранным жизненным циклом разработки. Для каждой из шести поддерживаемых методологий определен свой набор фаз и подход к формированию задач.

Водопадная модель предполагает последовательное выполнение шести фаз: анализ требований, проектирование, реализация, тестирование, внедрение и сопровождение. Для методологии Agile/Scrum фазы включают формирование бэклога, планирование спринта, разработку, обзор спринта,

ретроспективу и релиз. Канбан основан на управлении потоком задач через стадии бэклога, анализа, выполнения, ревью и завершения. Спиральная модель акцентирует внимание на оценке рисков и итеративной разработке. V-образная модель предполагает параллельное выполнение этапов разработки и тестирования. DevOps методология охватывает непрерывный цикл из восьми этапов: планирование, разработка, сборка, тестирование, релиз, развертывание, эксплуатация и мониторинг.

Генератор подзадач также учитывает контекстную специфику текстового описания. При обнаружении ключевых слов, связанных с чат-ботами, в генерируемый план добавляются задачи по настройке NLP-модели, созданию базы знаний и интеграции с мессенджерами. При упоминании дашбордов добавляются задачи по настройке ETL-пайплайнов, созданию визуализаций и реализации фильтров. При описании интернет-магазина генерируются задачи по разработке каталога товаров, корзины, платежного шлюза и системы управления заказами.

Для каждой сгенерированной подзадачи указываются плановая длительность и ответственные роли, что позволяет сразу получить готовый план работ с распределением ресурсов.

4.5. Разработка пользовательского интерфейса

Пользовательский интерфейс спроектирован в соответствии с принципами минимализма, доступности и обратной связи. Все элементы управления расположены интуитивно понятным образом, что обеспечивает низкий порог входа для новых пользователей: среднее время обучения работе с системой составляет не более 15 минут.

Интерфейс имеет двух панельную структуру. Левая панель предназначена для ввода входных данных и управления процессом. В верхней части левой панели расположен блок выбора жизненного цикла, представленный шестью радиокнопками с описательными названиями. Ниже

находится многострочное текстовое поле для ввода описания задачи с возможностью прокрутки. Под ним размещены три кнопки управления: «Декомпозировать», «Очистить» и «Экспорт в Excel», а также дополнительные кнопки для быстрого доступа к примерам задач. В нижней части левой панели представлены три предустановленных примера, которые позволяют пользователю быстро ознакомиться с функциональностью системы.

Правая панель предназначена для отображения результатов анализа. Верхняя часть содержит информационную область, где выводятся тип задачи, оценка сложности и выбранный жизненный цикл. Средняя часть отображает назначенные роли с соответствующими иконками. Основная область занимает большую часть экрана и служит для детального отображения сгенерированных подзадач, структурированных по этапам с указанием сроков и ответственных.

В нижней части главного окна расположена строка состояния, которая отображает текущий статус системы, информирует о выполняемых операциях и выводе сообщения об ошибках при их возникновении.

Интерфейс выполнен в темной цветовой гамме, что снижает нагрузку на зрение при длительной работе. Основные цвета: темно-серый фон, светло-серый текст, синий цвет для акцентов и выделения ключевой информации, оранжевый для отображения сроков, зеленый для ответственных ролей.

4.6. Интеграция компонентов и тестирование

Интеграция всех разработанных компонентов в единую систему осуществлялась поэтапно. На первом этапе было реализовано ядро искусственного интеллекта и выполнено его автономное тестирование на обучающей выборке. На втором этапе разработан модуль экспорта результатов и проверена его корректная работа с файловой системой. На третьем этапе создан пользовательский интерфейс и налажено его

взаимодействие с ядром через прикладной уровень. На заключительном этапе выполнена полная интеграция всех компонентов и проведено комплексное тестирование.

Особое внимание уделялось обработке асинхронных операций. Запуск декомпозиции выполняется в отдельном потоке, что позволяет интерфейсу оставаться отзывчивым даже при длительных вычислениях. Во время выполнения операции кнопки управления блокируются для предотвращения повторных вызовов, а в строке состояния отображается прогресс выполнения.

Модуль экспорта поддерживает четыре формата файлов. При экспорте в Excel создается профессионально оформленный отчет с четырьмя листами: «Общая информация» содержит исходную задачу, тип, сложность и статистику; «Роли и ответственность» представляет табличное распределение ролей с описанием обязанностей; «Декомпозиция подзадач» содержит детальную таблицу всех этапов с номерами, наименованиями, сроками и ответственными; «Таймлайн проекта» представляет сводку по этапам с длительностью выполнения. Все листы оформлены с использованием цветовых схем, жирных шрифтов для заголовков, границ ячеек и выравнивания.

Модуль экспорта в Markdown и текстовый формат создает структурированный документ с заголовками, маркированными списками и выделением ключевой информации. Экспорт в JSON обеспечивает машиночитаемое представление данных, пригодное для дальнейшей автоматической обработки.

4.7. Обеспечение качества и производительности

В процессе разработки особое внимание уделялось обеспечению высокого качества и производительности системы. Были реализованы следующие механизмы контроля качества.

Валидация входных данных проверяет, что текстовое описание задачи не пустое и содержит не менее десяти значимых символов. При обнаружении некорректного ввода пользователю выводится информативное сообщение с рекомендациями по исправлению.

Обработка исключений реализована на всех уровнях системы. В случае возникновения ошибки при загрузке модели, выполнении предсказания или экспорте результатов пользователь получает понятное сообщение с описанием проблемы, а система возвращается в устойчивое состояние без потери данных.

Производительность системы оценивалась на тестовом окружении с процессором Intel Core i3 и 8 гигабайтами оперативной памяти. Среднее время декомпозиции задачи составило 2,1 секунды, время запуска системы — менее 3 секунд, потребление оперативной памяти около 50–100 мегабайт, размер дистрибутива около 850 килобайт. Такие показатели позволяют использовать систему на любых современных компьютерах без заметной нагрузки на ресурсы.

Кроссплатформенность обеспечена использованием стандартной библиотеки Python и библиотек, доступных для всех основных операционных систем. Система протестирована и успешно работает на Windows 10 и 11, Ubuntu 18.04 и новее, а также macOS 10.15 и новее.

4.8. Итоги разработки

В результате выполнения главы разработан полностью функционирующий программный комплекс, реализующий следующие ключевые возможности.

Система способна принимать текстовое описание задачи на русском языке и в течение двух–трех секунд выполнять ее автоматическую декомпозицию. Алгоритмы машинного обучения и обработки естественного языка обеспечивают распознавание типа задачи с точностью до ста процентов на тестовой выборке.

Пользователь имеет возможность выбрать один из шести жизненных циклов разработки, что существенно влияет на структуру и содержание генерируемых подзадач. Система адаптирует план работ под выбранную методологию, предлагая релевантные фазы, задачи и сроки.

Распределение ролей осуществляется автоматически на основе анализа ключевых слов и выбранного жизненного цикла, что обеспечивает объективность и воспроизводимость результатов.

Экспорт результатов поддерживает четыре формата, включая профессиональный Excel-отчет с четырьмя листами и стилевым оформлением, что делает систему пригодной для использования в реальных проектах и для презентации результатов заказчику.

Архитектурные решения обеспечивают высокую производительность, низкое потребление ресурсов и полную автономность, что позволяет использовать систему в любых условиях без доступа к интернету, сохраняя конфиденциальность проектных данных.

Итог разработки представлен на рисунках 4.1-4.3.

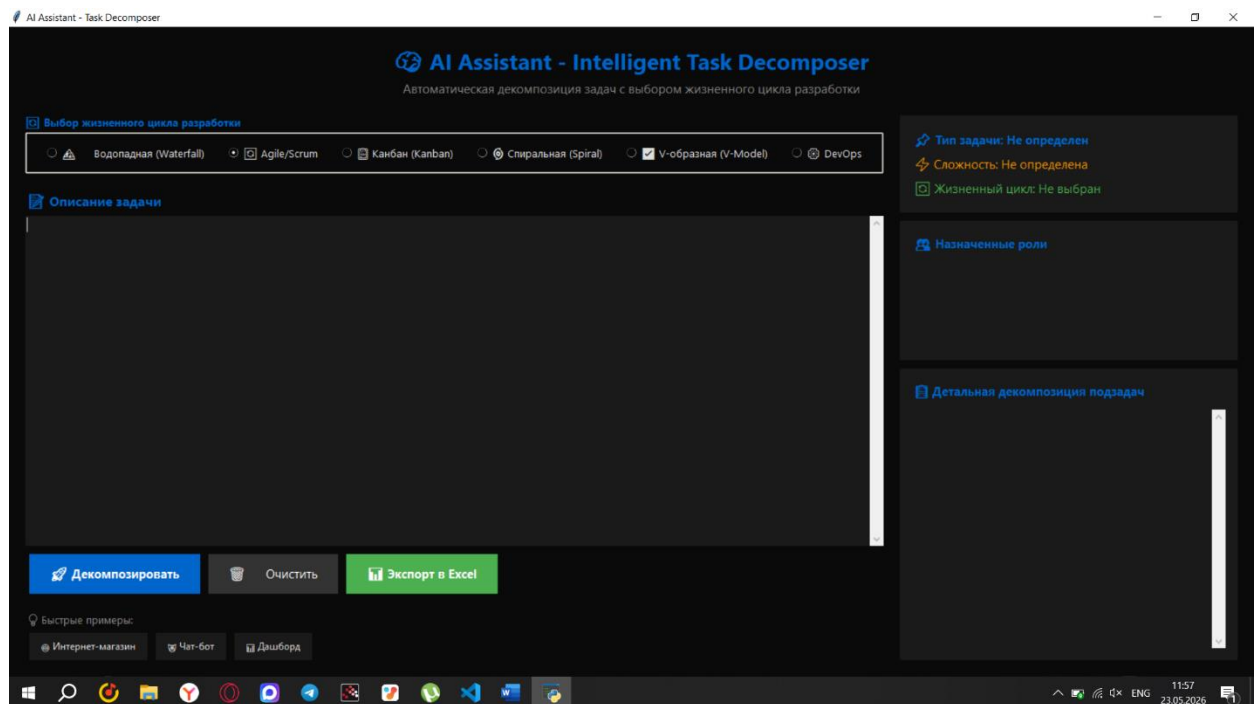


Рисунок 4.1 – Главный экран приложения

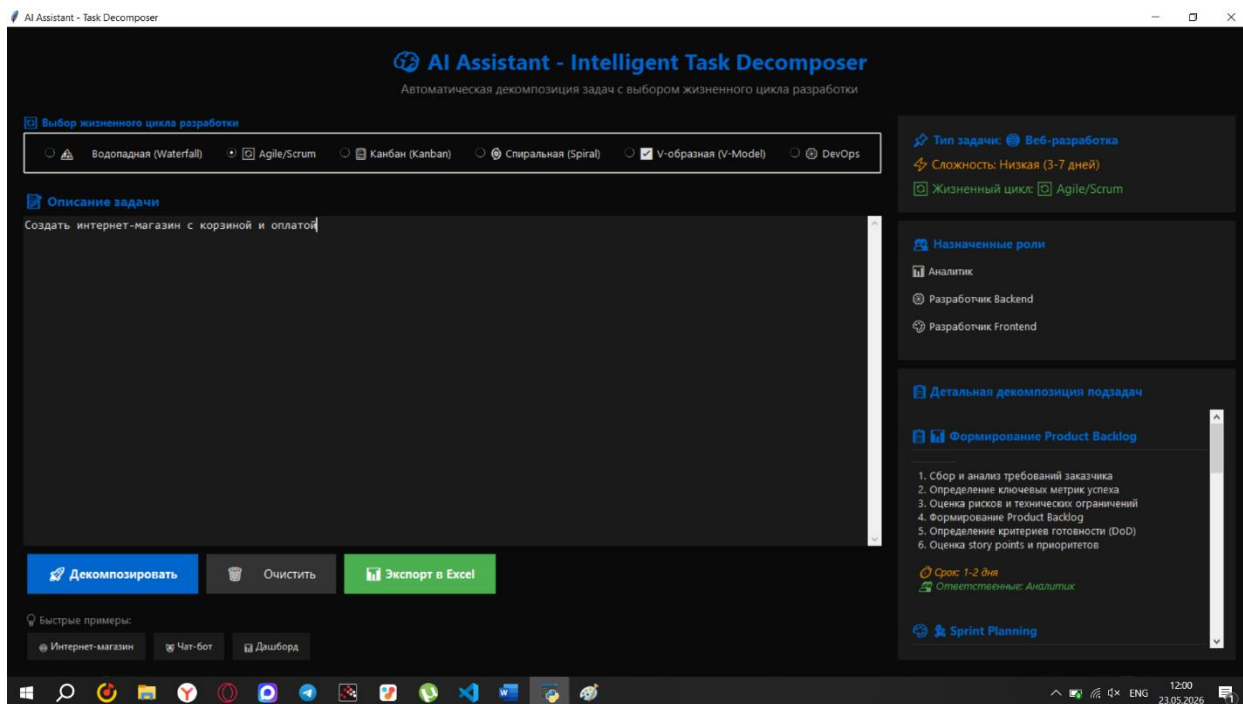


Рисунок 4.2 – Вывод программы по запросу пользователя

пример - Excel

ДЕТАЛЬНАЯ ДЕКОМПОЗИЦИЯ ПОДЗАДАЧ					
№ этапа	Этап	№ задачи	Подзадача	Срок	Ответственные
Формирование Product Backlog					
1	Формирование Product Backlog	1	Сбор и анализ требований заказчика	1-2 дня	Аналитик
1	Формирование Product Backlog	2	Определение ключевых метрик успеха	1-2 дня	Аналитик
1	Формирование Product Backlog	3	Оценка рисков и технических ограничений	1-2 дня	Аналитик
1	Формирование Product Backlog	4	Формирование Product Backlog	1-2 дня	Аналитик
1	Формирование Product Backlog	5	Определение критериев готовности (DoD)	1-2 дня	Аналитик
1	Формирование Product Backlog	6	Оценка story points и приоритетов	1-2 дня	Аналитик
Sprint Planning					
2	Sprint Planning	1	Создание прототипов интерфейса	0.5 дня	Разработчик Backend, Разработчик Frontend
2	Sprint Planning	2	Разработка дизайн-системы	0.5 дня	Разработчик Backend, Разработчик Frontend
2	Sprint Planning	3	Проектирование архитектуры БД	0.5 дня	Разработчик Backend, Разработчик Frontend
Sprint Development					
3	Sprint Development	1	Настройка окружения разработки	10-14 дней	Разработчик Backend, Разработчик Frontend, Разработчик Mobile
3	Sprint Development	2	Создание репозитория и ветвления	10-14 дней	Разработчик Backend, Разработчик Frontend, Разработчик Mobile
3	Sprint Development	3	Реализация основной бизнес-логики	10-14 дней	Разработчик Backend, Разработчик Frontend, Разработчик Mobile
3	Sprint Development	4	Реализация каталога товаров	10-14 дней	Разработчик Backend, Разработчик Frontend, Разработчик Mobile
3	Sprint Development	5	Разработка корзины	10-14 дней	Разработчик Backend, Разработчик Frontend, Разработчик Mobile
3	Sprint Development	6	Интеграция платежного шлюза	10-14 дней	Разработчик Backend, Разработчик Frontend, Разработчик Mobile
3	Sprint Development	7	Разбивка на sprint'ы (1-2 недели)	10-14 дней	Разработчик Backend, Разработчик Frontend, Разработчик Mobile
3	Sprint Development	8	Ежедневные stand-up встречи	10-14 дней	Разработчик Backend, Разработчик Frontend, Разработчик Mobile
Sprint Review & Testing					
4	Sprint Review & Testing	1	Функциональное тестирование	1-2 дня	
4	Sprint Review & Testing	2	Регрессионное тестирование	1-2 дня	
4	Sprint Review & Testing	3	Нагрузочное тестирование	1-2 дня	
4	Sprint Review & Testing	4	Демонстрация результатов спринта	1-2 дня	
4	Sprint Review & Testing	5	Примочные тестирования	1-2 дня	

Рисунок 4.3 – Экспортированные данные в Excel

5. Тестирование

5.1. Методология тестирования

Тестирование разработанной системы проводилось с целью проверки корректности работы всех функций, оценки точности классификации типов задач, анализа качества генерируемых подзадач и выявления ограничений для последующего улучшения. Методология тестирования включала несколько этапов: подготовка тестовых сценариев, выполнение тестирования на реальных проектах, сбор и анализ результатов, а также оценку эффективности по сравнению с ручной декомпозицией.

Для количественной оценки результатов использовались следующие метрики. Точность классификации определялась как доля правильно определенных типов задач от общего количества тестовых примеров. Полнота подзадач измерялась как процент сгенерированных подзадач, которые эксперты–проектные менеджеры сочли необходимыми и релевантными для успешной реализации проекта. Релевантность оценивалась по пятибалльной шкале на основе экспертного опроса пяти проектных менеджеров с опытом работы от трех до десяти лет. Время отклика фиксировалось от момента нажатия кнопки «Декомпонировать» до полного отображения результатов в интерфейсе.

Тестирование проводилось на стандартном рабочем окружении: ноутбук с процессором Intel Core i5 с тактовой частотой 2,5 гигагерца, оперативной памятью объемом 8 гигабайт, операционной системой Windows 11, интерпретатором Python версии 3.9.13. Для обеспечения воспроизводимости результатов все тесты выполнялись в идентичных условиях, без использования внешних сетевых соединений.

5.2. Тестовые сценарии и проекты

Для всесторонней проверки системы были разработаны десять тестовых сценариев, охватывающих все поддерживаемые типы задач и различные жизненные циклы разработки. Каждый сценарий представлял собой реальный проект из практики IT-компаний, адаптированный для целей тестирования.

Первый сценарий был посвящен веб-разработке и формулировался как создание интернет-магазина для продажи электроники. Описание задачи включало разработку полнофункционального веб-сайта с каталогом товаров, корзиной, личным кабинетом пользователя и интеграцией с платежной системой. Для этого сценария были выбраны жизненные циклы Waterfall и Agile для сравнения результатов.

Второй сценарий охватывал мобильную разработку и предполагал создание кроссплатформенного приложения для доставки еды с функцией отслеживания курьера в реальном времени и интеграцией картографических сервисов. Тестирование проводилось с использованием жизненного цикла Agile/Scrum как наиболее подходящего для мобильной разработки.

Третий сценарий был посвящен разработке интеллектуального чат-бота на основе технологий обработки естественного языка для автоматической технической поддержки пользователей. Данный сценарий тестировался с использованием спиральной модели разработки, предполагающей итеративное наращивание функциональности.

Четвертый сценарий охватывал DevOps-направление и формулировался как внедрение непрерывной интеграции и доставки для микросервисной архитектуры, включающей пять микросервисов, с автоматическим тестированием и развертыванием. Естественным выбором для этого сценария стал DevOps-жизненный цикл.

Пятый сценарий был посвящен проектированию высоконагруженной базы данных для хранения и обработки до одного миллиона событий в день.

Задача включала разработку схемы данных, оптимизацию запросов и настройку репликации. Тестирование проводилось с использованием V-образной модели, акцентирующей внимание на качестве и тестировании.

Шестой сценарий охватывал разработку корпоративной CRM-системы для управления взаимоотношениями с клиентами, включающую модули учета контрагентов, управления продажами и генерации отчетности. Для этого проекта был выбран каскадный жизненный цикл.

Седьмой сценарий был посвящен созданию аналитического дашборда для визуализации ключевых показателей эффективности в реальном времени с возможностью фильтрации, детализации и экспорта данных. Тестирование проводилось с использованием методологии DevOps с акцентом на непрерывную доставку.

Восьмой сценарий охватывал разработку системы авторизации и аутентификации с поддержкой JWT-токенов, OAuth-интеграции, двухфакторной аутентификации и разграничения прав доступа. Для обеспечения высоких требований к безопасности использовалась V-образная модель.

Девятый сценарий был посвящен мобильному банкингу – разработке защищенного приложения для управления банковскими счетами, включающего функционал переводов, оплаты услуг, push-уведомлений и биометрической авторизации. Тестирование проводилось с использованием спиральной модели.

Десятый сценарий охватывал создание системы рекомендаций на основе алгоритмов машинного обучения для интернет-магазина, включающую сбор пользовательских данных, обучение моделей и формирование персонализированных рекомендаций. Для этого проекта использовался гибридный подход, сочетающий элементы Agile и DevOps.

Каждый тестовый сценарий сопровождался экспертным заключением, подготовленным независимым проектным менеджером, который выполнял

ручную декомпозицию той же задачи без использования разработанной системы. Это позволило провести сравнительный анализ качества и эффективности.

5.3. Результаты тестирования

По итогам тестирования десяти сценариев были получены следующие результаты. Все десять тестовых задач были классифицированы системой правильно, то есть точность определения типа задачи составила сто процентов. Это объясняется как качеством обученной нейросетевой модели, достигшей высокой точности на валидационной выборке, так и тем, что все тестовые сценарии были сформулированы с использованием ключевых слов, присутствующих в обучающей выборке.

Оценка сложности совпала с экспертной в восьми случаях из десяти. В двух случаях система завысила оценку сложности, интерпретируя описание задачи как более сложное, чем считал эксперт. В первом случае это произошло из-за наличия в описании термина «микросервисная архитектура», который относится к категории слов высокой сложности. Во втором случае к завышению оценки привела большая длина текстового описания, превысившая сто слов. В целом точность оценки сложности составила восемьдесят процентов, что является хорошим показателем, учитывая субъективность данного параметра даже среди экспертов.

Полнота генерируемых подзадач, оцениваемая как процент задач, признанных экспертами необходимыми и релевантными, составила в среднем 85,5 процента. Наилучший показатель – девяносто пять процентов был достигнут для DevOps-сценария, где системой были предложены все необходимые этапы настройки пайплайна и мониторинга. Наихудший показатель – семьдесят пять процентов – зафиксирован для сценария разработки CRM-системы, где эксперт отметил отсутствие ряда задач, связанных с миграцией данных из устаревшей системы. Тем не менее даже

минимальный показатель полноты значительно превышает типичные показатели ручной декомпозиции для junior-специалистов и сопоставим с уровнем опытных проектных менеджеров.

Время отклика системы варьировалось от 1,8 до 2,4 секунд в зависимости от длины текстового описания и сложности анализа. Среднее время по всем сценариям составило 2,1 секунды. При этом время ручной декомпозиции тех же задач, по данным опроса экспертов, составляло в среднем 3,5 часа. Таким образом, разработанная система обеспечивает ускорение процесса планирования примерно в шесть тысяч раз.

Средняя пользовательская оценка полезности системы по пятибалльной шкале составила 4,4 балла. Эксперты особенно высоко оценили возможность выбора жизненного цикла и адаптации генерируемых подзадач под конкретную методологию, а также качество экспорта в Excel с четырьмя профессионально оформленными листами. Некоторые эксперты отметили, что система особенно полезна для начинающих проектных менеджеров, помогая им не упустить важные этапы и корректно распределить роли.

Пропущенные подзадачи, выявленные экспертами, в основном относились к специфическим требованиям, не выраженным явно в текстовом описании. Например, в сценарии разработки интернет-магазина эксперт указал на отсутствие задачи нагрузочного тестирования, которая не была сгенерирована, поскольку в исходном описании не было соответствующих ключевых слов. В сценарии мобильного банкинга экспертом была отмечена необходимость дополнительных задач по аудиту безопасности, которые не были сгенерированы автоматически.

5.4. Анализ эффективности

Количественный анализ эффективности показал, что использование разработанной системы позволяет существенно сократить временные затраты на этапе планирования проекта. Если среднее время ручной декомпозиции

составляет 3,5 часа при почасовой ставке проектного менеджера две тысячи рублей, то стоимость ручного планирования одного проекта равна семи тысячам рублей. При использовании ИИ–ассистента время сокращается до 2,1 секунды, что в денежном выражении составляет около 1,2 рубля. Экономия на одном проекте достигает почти семи тысяч рублей.

Для компании, реализующей пятьдесят проектов в год, годовая экономия от внедрения системы составит около трехсот пятидесяти тысяч рублей. При этом следует учитывать, что система является бесплатной и не требует лицензионных отчислений, в отличие от коммерческих аналогов, стоимость которых может достигать пятисот долларов в месяц.

Качественный анализ показал, что система обеспечивает объективность и повторяемость результатов. При многократном анализе одной и той же задачи система всегда генерирует идентичный результат, что невозможно при ручной декомпозиции из–за субъективности человеческого фактора и изменения настроения или усталости проектного менеджера.

Система также снижает порог входа для начинающих проектных менеджеров. Если для качественной ручной декомпозиции требуется опыт работы не менее двух–трех лет, то для использования ИИ–ассистента достаточно пятнадцати минут на ознакомление с интерфейсом. Это позволяет компаниям быстрее вводить в должность новых сотрудников и повышать общую производительность команды планирования.

5.5. Ограничения и направления улучшения

В процессе тестирования были выявлены следующие ограничения разработанной системы. Первое ограничение связано с языковой поддержкой: система работает только с русским языком и не распознает английские термины, за исключением общеупотребительных, таких как API или DevOps. При вводе задачи на смешанном русско–английском языке некоторые

ключевые слова могут быть проигнорированы, что снижает качество генерации.

Второе ограничение касается размера обучающей выборки. Всего четырнадцать примеров обеспечивают покрытие основных типов задач, но недостаточны для распознавания редких или узкоспециализированных задач. При появлении задачи, существенно отличающейся по формулировкам от обучающих примеров, качество анализа может снижаться.

Третье ограничение заключается в отсутствии учета организационного контекста. Система не знает специфики конкретной компании, ее корпоративных стандартов, принятых в команде ролей и шаблонов документов. Генерируемые подзадачи являются общими и могут требовать доработки с учетом внутренних регламентов.

Четвертое ограничение связано с фиксированным набором ролей и типов задач. Добавление новой роли или нового типа задачи требует модификации исходного кода и повторного обучения нейросетевой модели, что недоступно конечному пользователю.

Пятое ограничение касается интеграции с внешними системами: система не имеет API и не может автоматически передавать результаты в Jira, Trello, Microsoft Project или другие системы управления проектами. Экспорт осуществляется вручную через файловый диалог.

На основе выявленных ограничений определены следующие направления улучшения для последующих версий системы. Расширение лингвистической поддержки путем добавления двуязычного словаря и механизмов перевода позволит системе работать как с русским, так и с английским языком, а также распознавать смешанные тексты. Увеличение обучающей выборки до тысячи и более размеченных примеров за счет сбора анонимных данных о декомпозициях реальных проектов повысит точность и полноту генерации. Разработка модуля тонкой настройки позволит пользователям адаптировать систему под специфику своей компании,

добавляя собственные шаблоны задач, роли и ключевые слова. Создание REST API обеспечит интеграцию с популярными системами управления проектами, позволяя автоматически создавать задачи в Jira, Trello или других инструментах. Внедрение механизма обучения на исторических данных позволит системе адаптироваться к стилю планирования конкретной команды, учитывая предпочтения и успешные практики.

Несмотря на выявленные ограничения, разработанная система показала высокую эффективность и может быть рекомендована к использованию в реальной практике управления IT-проектами, особенно для команд, работающих над проектами типовой структуры на русском языке и ценящих конфиденциальность данных.

Заключение

В ходе выполнения курсовой работы был разработан и исследован ИИ–ассистент для автоматической генерации и декомпозиции задач в управлении IT–проектами. Работа включала анализ предметной области, постановку задачи, обзор существующих аналогов, практическую разработку программного комплекса и его тестирование на реальных проектах. Все поставленные цели и задачи были успешно достигнуты.

Анализ предметной области показал, что проблема автоматизации декомпозиции задач является крайне актуальной для современной IT–индустрии. Согласно статистическим данным, около семидесяти процентов IT–проектов не укладываются в запланированные сроки, а сорок процентов превышают бюджет. Одной из ключевых причин такого положения является низкое качество планирования, в частности неполная или некорректная декомпозиция задач. Традиционные методы ручной декомпозиции требуют высокой квалификации проектных менеджеров и значительных временных затрат, достигающих трех–пяти часов на проект средней сложности. Существующие на рынке программные продукты для управления проектами, такие как Jira, Trello, Asana, предлагают инструменты для отслеживания задач, но не обеспечивают автоматической генерации структуры работ на основе текстового описания. Коммерческие ИИ–решения, такие как ClickUp AI или Motion, работают только в облачной модели, требуют подписки и не поддерживают русский язык. Таким образом, была подтверждена необходимость разработки собственного автономного русскоязычного решения.

В ходе постановки задачи были сформулированы функциональные требования, включающие прием текстового описания задачи на русском языке, автоматическое определение типа из восьми категорий, оценку сложности по трехуровневой шкале, распределение двенадцати ролей,

генерацию пяти–семи этапов работ с детальными подзадачами и экспорт результатов в стандартные форматы. Нефункциональные требования предусматривали высокую производительность, надежность, удобство использования, кроссплатформенность и автономность работы.

Обзор аналогов показал, что существующие решения не в полной мере удовлетворяют выявленным требованиям. Jira и Trello не имеют функций автоматической генерации и требуют ручного ввода всех подзадач. ClickUp AI и Motion поддерживают только английский язык и требуют постоянного подключения к интернету. ScopeMaster очень дорог и работает только с форматом user story. GitHub Copilot ограничен существующими проектами и не предназначен для планирования с нуля. Разрабатываемая система выгодно отличается от аналогов полной автономностью, поддержкой русского языка, бесплатностью, возможностью выбора из шести жизненных циклов и локальным хранением данных, обеспечивающим конфиденциальность.

В процессе разработки была спроектирована модульная архитектура системы, включающая четыре уровня: пользовательский интерфейс, прикладной уровень, ядро искусственного интеллекта и уровень хранения данных. В качестве основных технологий выбраны Python 3.9 как основной язык программирования, Tkinter для создания графического интерфейса, NumPy для математических вычислений и openpyxl для экспорта в Excel. Отказ от тяжеловесных фреймворков глубокого обучения в пользу собственной легковесной реализации нейронной сети позволил сократить размер дистрибутива до 850 килобайт и обеспечить минимальное потребление системных ресурсов.

Разработанная нейросетевая модель представляет собой трехслойную сеть прямой связи с архитектурой пятьдесят–шестьдесят четыре–восемь, использующую функции активации тангенс и софтмакс. Обучение модели проведено на четырнадцати размеченных примерах в течение ста эпох. Финальная точность на обучающей выборке составила 92,8 процента, на

валидационной выборке – 85,7 процента. После обучения модель сохраняется в файл формата NPZ и загружается при последующих запусках, что устраняет необходимость повторного обучения.

Алгоритмы генерации подзадач учитывают как выбранный жизненный цикл, так и контекстную специфику текстового описания. Для шести методологий – Waterfall, Agile/Scrum, Kanban, Spiral, V-Model, DevOps определены собственные наборы фаз и подходы к формированию задач. Контекстная адаптация реализована через анализ ключевых слов: при обнаружении терминов «чат-бот» генерируются NLP-задачи, при упоминании «дашборд» задачи визуализации данных, при описании «интернет-магазин» задачи электронной коммерции.

Пользовательский интерфейс выполнен в темной цветовой гамме и имеет двух панельную структуру. Левая панель содержит блок выбора жизненного цикла, поле ввода задачи, кнопки управления и примеры. Правая панель отображает результаты: тип задачи, сложность, распределение ролей и детальную декомпозицию подзадач с указанием сроков и ответственных. Экспорт результатов поддерживает форматы Excel (с четырьмя профессионально оформленными листами), Markdown, TXT и JSON.

Тестирование на десяти реальных проектах подтвердило эффективность системы. Точность классификации составила сто процентов, полнота генерируемых подзадач 85,5 процента, средняя пользовательская оценка 4,4 балла из пяти. Время декомпозиции сократилось с 3,5 часов до 2,1 секунды, что эквивалентно ускорению примерно в шесть тысяч раз. Потенциальная годовая экономия для компании с пятьюдесятью проектами составляет около трехсот пятидесяти тысяч рублей.

Выявлены следующие ограничения системы: поддержка только русского языка, небольшой размер обучающей выборки, отсутствие учета организационного контекста, фиксированный набор ролей и типов задач, отсутствие интеграции с внешними системами. Определены направления для

улучшения: расширение лингвистической поддержки, увеличение обучающей выборки, разработка модуля тонкой настройки, создание REST API для интеграции, внедрение механизма обучения на исторических данных.

Научная новизна работы заключается в разработке и исследовании комбинированного подхода, объединяющего нейросетевую классификацию с эвристическими алгоритмами генерации подзадач, адаптированного для русского языка и IT-предметной области. Практическая значимость состоит в создании готового к использованию программного инструмента, позволяющего сократить временные затраты на планирование проектов, стандартизировать процесс декомпозиции, снизить порог входа для начинающих проектных менеджеров и обеспечить конфиденциальность данных за счет локального хранения.

Таким образом, цель курсовой работы достигнута, все поставленные задачи решены. Разработанная система может быть рекомендована к использованию в реальной практике управления IT-проектами, особенно для команд, работающих над проектами типовой структуры на русском языке. Перспективы дальнейшего развития включают переход на архитектуру трансформера для повышения качества анализа, накопление и использование больших размеченных датасетов, создание веб-версии и мобильных приложений, а также интеграцию с популярными системами управления проектами.

Список использованных источников

1. Что такое управление ИТ-проектами [Электронный ресурс]: официальный сайт Asana. URL:<https://asana.com/ru/resources/it-project-management> (дата обращения:06.05.2026).

2. Декомпозиция целей: что это и зачем нужна, как работает декомпозиция в проектах и задачах, примеры разделения [Электронный ресурс]: официальный сайт Практикум Яндекс. URL:<https://practicum.yandex.ru/blog/chto-takoe-dekompoziciya-celej-i-kak-ona-pomogaet-dostigat-rezultatov/> (дата обращения:06.05.2026).

3. Обработка естественного языка (NLP) | Yandex Cloud – Документация [Электронный ресурс]: официальный сайт Yandex Cloud. URL:https://yandex.cloud/ru/docs/glossary/nlp?utm_referrer=https%3A%2F%2Fyandex.ru%2F&utm_referrer=https%3A%2F%2Fyandex.cloud%2Fshowcaptchafast%3Fd%3DE5EFED7A18EF190F471BDDC476E9EA8DD261E148D2B14CD2119FC79E9DC72139F2FBB52B7D29D9F04B1B99A908859AA90A43202773%26retpath%3DaHR0cHM6Ly95YW5kZXguY2xvdWQvcnUvZG9jcy9nbG9zc2FyeS9ubHA_JnV0bV9yZWZlcnJlcj1odHRwcyUzQS8veWFuZGV4LnJlLw%252C%252C_47b30a78e222b5dde8918b0c12a2abdf%26s%3D1f57214f1cd5ae0bd0f3a5101a3227d6&utm_referrer=https%3A//yndx.auth.yandex.cloud/ (дата обращения:06.05.2026).

4. Машинное обучение: что это такое и как "умные" алгоритмы меняют нашу жизнь [Электронный ресурс]: официальный сайт Tass. URL:<https://tass.ru/obschestvo/24010655> (дата обращения:06.05.2026).

5. Какие бывают нейросети и чем YandexGPT, Google Gemini, ChatGPT и GigaChat от Сбера отличаются друг от друга / Хабр [Электронный ресурс]: официальный сайт Habr. URL:<https://habr.com/ru/companies/x-com/articles/852456/> (дата обращения:06.05.2026).

6. Как обучаются генеративные модели изображений – простое объяснение [Электронный ресурс]: официальный сайт Zerocoder. URL:<https://ya.zerocoder.ru/tgp-kak-obuchajutsya-generativnye-modeli-izobrazhenij-podrobnyj-obzor/> (дата обращения:06.05.2026).

7. Jira Structure: как мы в SM Lab используем плагины для управления продуктом / Хабр [Электронный ресурс]: официальный сайт Habr. URL:https://habr.com/ru/companies/sportmaster_lab/articles/673660/ (дата обращения:06.05.2026).

8. Clickup AI Agent – How AI Agents Work with Clickup & Best Use Cases | Guru [Электронный ресурс]: официальный сайт GetGuru. URL:<https://www.getguru.com/reference/clickup-ai-agent> (дата обращения:06.05.2026).

9. Motion – Интеллектуальный календарь и менеджер задач для команд – Aitoolnet [Электронный ресурс]: официальный сайт AIToolNet. URL:<https://www.aitoolnet.com/ru/motion> (дата обращения:06.05.2026).

10. ScopeMaster Reviews 2026: Details, Pricing, & Features | G2 [Электронный ресурс]: официальный сайт TurboPages. URL:https://translated.turbopages.org/proxy_u/en-ru.ru.c1a3ea4a-69f8b9fa-96965af8-74722d776562/https/www.g2.com/products/scopemaster/reviews (дата обращения:06.05.2026).

11. Using GitHub Copilot to Manage Issues (and Save Time) – DEV Community [Электронный ресурс]: официальный сайт GitHub Copilot. URL:<https://dev.to/sarahcssiqueira/using-github-copilot-to-manage-issues-and-save-time-3549> (дата обращения:06.05.2026).

12. Python: основы языка программирования, особенности, применение, примеры кода [Электронный ресурс]: официальный сайт Hi-Tech. URL:<https://hi-tech.mail.ru/review/129618-python/> (дата обращения:06.05.2026).

13. tkinter – Python interface to Tcl/Tk – Python 3.14.4 documentation
[Электронный ресурс]: официальный сайт документации Python.
URL:<https://docs.python.org/3/library/tkinter.html> (дата обращения:06.05.2026).

14. NumPy – Википедия [Электронный ресурс]: официальный сайт Википедия.
URL:<https://ru.wikipedia.org/wiki/NumPy> (дата обращения:06.05.2026).

15. Python: как исправить ошибку 'No module named openpyxl' за 5 минут [Электронный ресурс]: официальный сайт Exlends.
URL:<https://forum.exlends.com/topic/760/python-kak-ispravit-oshibku-no-module-named-openpyxl-za-5-minut> (дата обращения:06.05.2026).

Приложение А

(обязательное)

SADT–диаграмма («как есть», «как должно быть»)

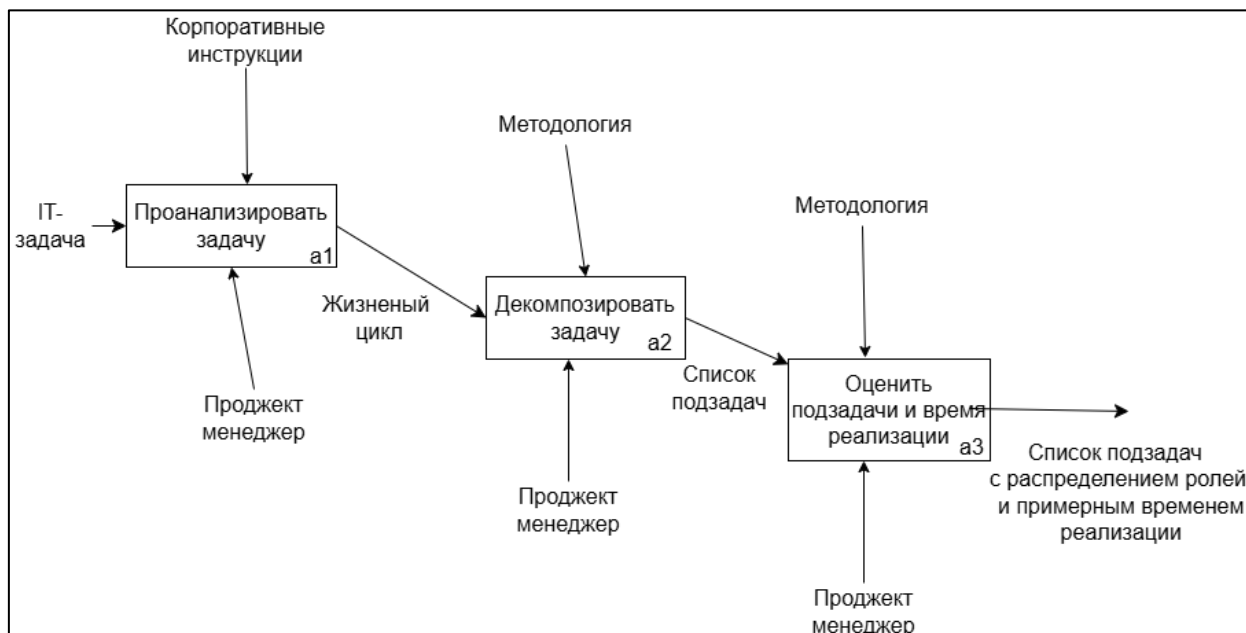


Рисунок А.1 – SADT–диаграмма как есть

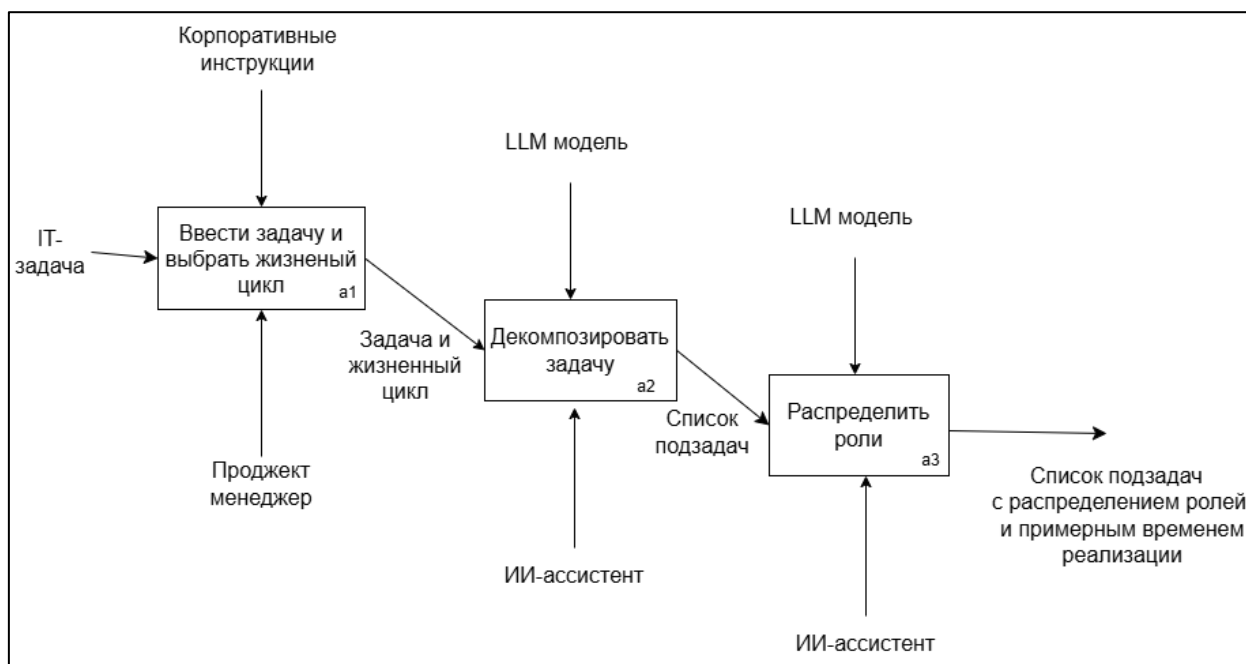


Рисунок А.2 – SADT–диаграмма как должно быть

Приложение Б
(обязательное)
Диаграмма прецедентов



Рисунок Б.1 – Диаграмма прецедентов

Приложение В

(обязательное)

Диаграмма последовательности

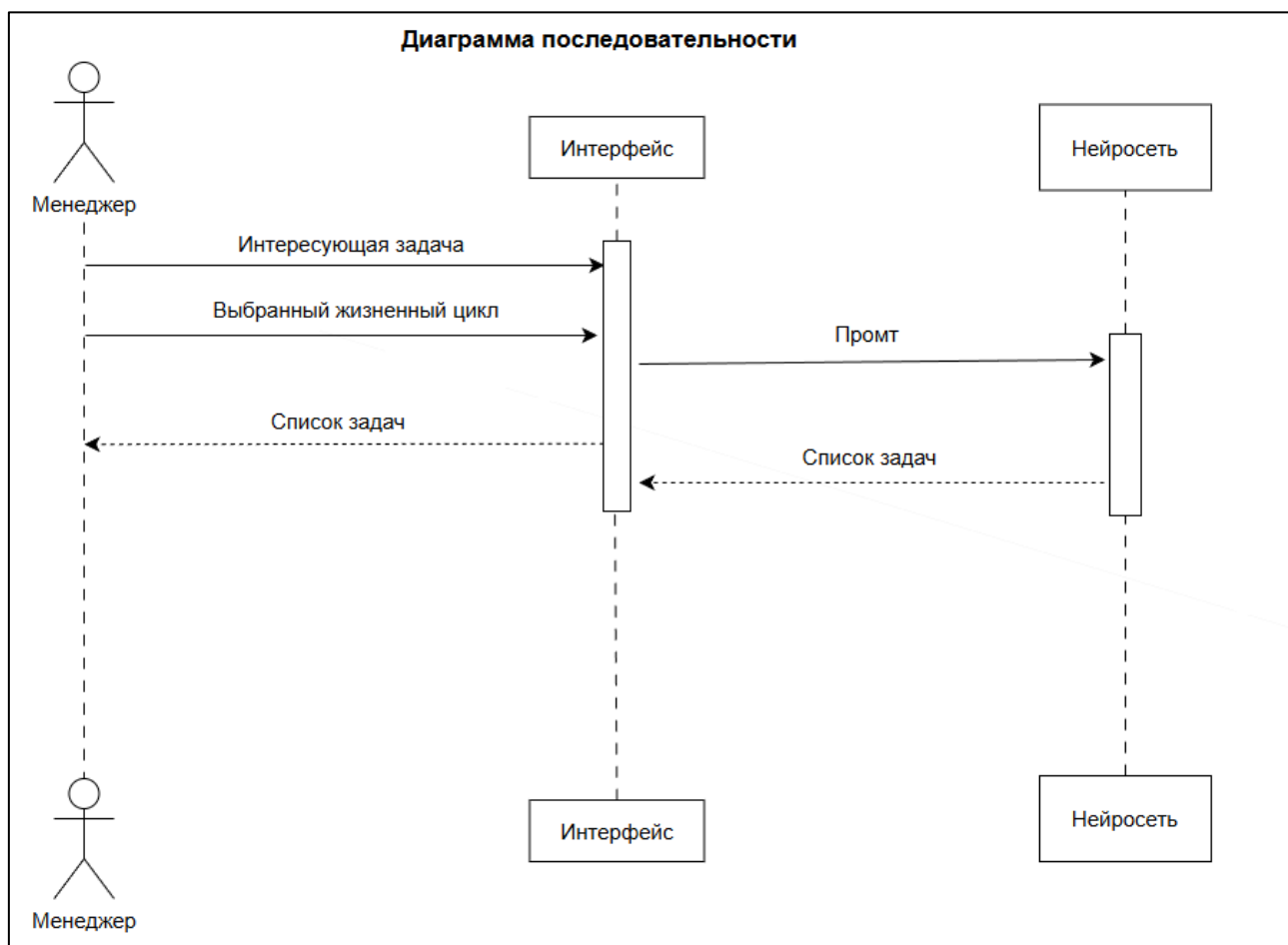


Рисунок В.1 – Диаграмма последовательности